Formal Software Verification

Project Report: SAT Solver



Authors:

Darius Mureşan
202502066

Rui Xavier
202502046

Teacher:
Bas Spitters

December 17, 2025

Contents

1 Introduction **2**

2 Formalizing Boolean Logic **2**
2.1 Syntax . . . 2
2.2 Semantics and Valuations . . . 3

3 Implementing a Decidable Solver **3**
3.1 The Problem of Infinite Search Spaces . . . 3
3.2 Generating Partial Valuations . . . 4
3.3 The Solver Function . . . 4

4 Verification: Soundness and Completeness **4**
4.1 Soundness . . . 5
4.2 Completeness . . . 5

5 Extensions and Advanced Features **6**
5.1 Three-Valued Logic . . . 6
5.2 Formula Optimization . . . 6
5.3 Normal Forms: NNF and CNF . . . 6
5.3.1 Negation Normal Form (NNF) . . . 6
5.3.2 Conjunctive Normal Form (CNF) . . . 7

6 Discussion and Conclusion **7**
6.1 Challenges and Solutions . . . 7
6.2 Experience with Rocq . . . 7
6.3 Conclusion . . . 8
6.4 Use of GAI . . . 8

A Appendix: Code Fragments **9**
A.1 Optimization Logic . . . 9
A.2 Distribution Logic for CNF . . . 9

1 Introduction

In the field of formal software verification, the reliability of tools used to verify other systems is paramount. One such fundamental tool is the Boolean Satisfiability (SAT) solver. The SAT problem asks whether there exists an assignment of truth values to variables in a Boolean formula such that the formula evaluates to true. Because SAT is NP-complete, efficient solvers are complex, and their complexity makes them prone to bugs.

This project focuses on the formalization and verification of a naive SAT solver using the Rocq (Coq) proof assistant. The objective is not to build the most efficient solver, but to build a *certified* one, a solver accompanied by machine-checked proofs of its correctness.

The project is divided into three main components. First, we define the syntax and semantics of Boolean logic within the constructive logic of Rocq. Second, we implement a computable decision procedure (a solver) that determines satisfiability by exhaustively searching the valuation space. Third, we verify this solver by proving two key properties: *soundness* (if the solver says "satisfiable", it really is) and *completeness* (if the formula is satisfiable, the solver will say so).

Furthermore, we have extended the project beyond the basic requirements to include three-valued logic, formula optimizations, and transformations into Negation Normal Form (NNF) and Conjunctive Normal Form (CNF). These extensions demonstrate how structural transformations can be verified to preserve semantics.

2 Formalizing Boolean Logic

To reason about satisfiability, we must first embed the language of Boolean logic into Rocq. This involves defining the abstract syntax tree of formulas and assigning meaning to them via operational semantics.

2.1 Syntax

The syntax of Boolean formulas is defined inductively. A formula can be a variable (identified by an ID), a constant (True or False), or a combination of formulas using standard logical connectives: negation ($\neg$), conjunction ($\wedge$), disjunction ($\vee$), and implication ($\rightarrow$).

Listing 1: Inductive definition of Boolean formulas

```
Inductive form :=
| var : id → form
| FTrue : form
| FFalse : form
| FNeg : form → form
| FConj : form → form → form
```

```
| FDisj : form → form → form
| FImpl : form → form → form.
```

This definition allows us to represent arbitrary Boolean expressions. For instance, the formula $(x \vee \neg y) \wedge (\neg x \vee y)$ is represented structurally within the type system.

2.2 Semantics and Valuations

The meaning of a Boolean formula is derived from a *valuation*. In classical logic, a valuation assigns a truth value to every possible variable. We modeled this in Rocq as a function mapping identifiers to booleans:

```
Definition valuation := id → bool.
```

Using this definition, the semantics are defined via an interpretation function, `interp`, which acts as a recursive evaluator. It folds over the inductive structure of the formula, reducing logical connectives to their boolean counterparts in Rocq (e.g., `FConj` maps to the boolean AND operator `&&`).

Definition 2.1 (Satisfiability)**.** *A formula p is satisfiable if and only if there exists a valuation V such that the interpretation of p under V is true.*

$$satisfiable(p) \triangleq \exists V : valuation, interp\ V\ p = true$$

This definition relies on the existential quantifier in Rocq's `Prop` sort. While mathematically precise, this definition is not directly computable because we cannot computationally search the infinite space of all functions of type `id -> bool`. This distinction between the logical definition and the computable implementation is a central theme of the project.

3 Implementing a Decidable Solver

The core challenge in this project was bridging the gap between the infinite domain of logical valuations and the finite requirements of a computable function.

3.1 The Problem of Infinite Search Spaces

A naive approach to SAT solving involves truth tables: checking every possible assignment. However, our definition of `valuation` is infinite. To create a terminating `Fixpoint` in Rocq, we must identify a finite search space.

I observed that the truth value of a formula p depends *only* on the variables that appear in p. The values assigned to variables not in p are irrelevant. Therefore, we implemented a function `vars` to extract the list of relevant identifiers from a formula.

3.2 Generating Partial Valuations

To simulate the truth table, we implemented a generator, `all_valuations`, which takes a list of relevant variables and produces a list of all distinct partial valuations for those variables.

Listing 2: Generating the search space

```
Fixpoint all_valuations (vars : list id) : list valuation :=
match vars with
| [] ⇒ [empty_valuation]
| x :: xs ⇒
let rest := all_valuations xs in
(map (λ V ⇒ override V x true) rest) ++
(map (λ V ⇒ override V x false) rest)
end.
```

This function works by induction on the list of variables. For the base case (empty list), it returns a single empty valuation. For the inductive step, it takes all valuations for the tail of the list and branches them: one version where variable x is true, and one where x is false.

This approach ensures that if the input list contains unique variables, the output list contains 2^n valuations, covering every possible truth assignment for the relevant variables.

3.3 The Solver Function

With the search space finitized, the solver itself is a straightforward search. The function `find_valuation` generates the search space for the variables in p and iterates through it.

```
Definition solver (p : form) : bool :=
match find_valuation p with
| Some _ ⇒ true
| None ⇒ false
end.
```

This function is guaranteed to terminate and returns a boolean, making it a valid decision procedure within Rocq.

4 Verification: Soundness and Completeness

Writing the program is only half the task; the core of the project is proving that `solver` precisely captures the notion of `satisfiable`.

4.1 Soundness

Soundness is the property that "if the solver returns true, the formula is satisfiable".

Theorem 4.1 (Soundness)**.** $\forall p$, *solver* $p = true \rightarrow$ *satisfiable* p.

Proof Strategy: The proof proceeds by inspecting the definition of `solver`. The solver only returns `true` if `find_valuation` returns `Some V`. By the definition of `find_valuation`, this V comes from the list of generated valuations and satisfies `interp V p = true`. Since we have constructed a specific witness V that satisfies the formula, we can simply provide this witness to the existential quantifier in the definition of `satisfiable`. This direction was straightforward to formalize.

4.2 Completeness

Completeness is the converse property: "if the formula is satisfiable, the solver returns true". This was the most difficult part of the verification process.

Theorem 4.2 (Completeness)**.** $\forall p$, *satisfiable* $p \rightarrow$ *solver* $p = true$.

The Challenge: The hypothesis `satisfiable p` gives us an arbitrary valuation V_{global}. This V_{global} is a function defined for all IDs. The solver, however, only checks the specific finite list of valuations generated by `all_valuations`. To prove completeness, we must show that inside that finite list, there exists a valuation V_{local} that behaves exactly like V_{global} with respect to the formula p.

Key Lemma 1: Interpretation Agreement
I first proved that if two valuations agree on all variables occurring in a formula, the formula evaluates to the same result under both.

$$\forall p\ V_1\ V_2, (\forall x \in \text{vars}(p), V_1\ x = V_2\ x) \rightarrow \text{interp}\ V_1\ p = \text{interp}\ V_2\ p$$

This lemma allows us to bridge the gap between the arbitrary V_{global} and our constructed V_{local}.

Key Lemma 2: Search Space Coverage
Next, we proved that `all_valuations` is exhaustive. Specifically, for any list of variables L and any arbitrary valuation V, there exists a valuation V' inside `all_valuations L` such that V and V' agree on all variables in L.

Listing 3: The Coverage Lemma

```
Lemma all_valuations_covers : ∀ l V,
∃V', In V' (all_valuations l) ∧
(∀ x, In x l → V' x = V x).
```

Proof of Completeness: 1. Assume p is satisfiable. This gives us a witness V_{global} such that interp V_{global} p = true. 2. Apply the Coverage Lemma to V_{global} and the variables of p. This proves there exists a V_{local} in our search space that matches V_{global} on relevant variables. 3. By the Agreement Lemma, since V_{local} and V_{global}

match on relevant variables, interp V_{local} p = interp V_{global} p = true. 4. Since V_{local} is in the search list and evaluates to true, the search function `find_sat` will find it (or another satisfying valuation appearing earlier in the list) and return `Some`. 5. Therefore, `solver p` returns `true`.

5 Extensions and Advanced Features

To demonstrate a deeper understanding of semantic preservation and structural transformations, we implemented three significant extensions to the base solver.

5.1 Three-Valued Logic

I extended the boolean logic to a ternary system with values {True, False, Unknown}. This is useful in scenarios where the value of a variable might not be determined.

- **Implementation:** We defined a new type `tri` and lifted the standard boolean operators. For example, `AND(True, Unknown)` evaluates to `Unknown`, whereas `AND(False, Unknown)` evaluates to `False` (short-circuiting).
- **Consistency:** We proved that this logic is consistent with standard boolean logic. Specifically, we defined a projection from `bool` to `tri` and proved that the standard interpretation is isomorphic to the ternary interpretation when the input valuation contains no unknowns.

5.2 Formula Optimization

I implemented a constant-folding optimizer. This function recursively traverses the formula and simplifies expressions involving constants (e.g., $p \wedge \text{True} \to p$, $p \wedge \text{False} \to$ False).

The verification obligation was to prove `interp V (optimize p) = interp V p`. This was achieved via structural induction. While the proof was relatively simple (mostly rewriting inductive hypotheses), the number of cases (commutativity, identity elements) made it verbose.

5.3 Normal Forms: NNF and CNF

The most complex extension was converting formulas into Conjunctive Normal Form (CNF). CNF is the standard input format for industrial SAT solvers.

5.3.1 Negation Normal Form (NNF)

First, we transformed formulas into NNF, where negation is only applied to variables. This requires pushing negations down using De Morgan's laws (e.g., $\neg(A \wedge B) \to \neg A \vee \neg B$). Because the transformation of $\neg p$ depends on the structure of p, the definition of `to_nnf` and its helper `neg_nnf` are mutually recursive. Consequently,

the proof of correctness required **mutual induction**, proving two properties simultaneously:

1. interp V (to_nnf p) = interp V p
2. interp V (neg_nnf p) = $\neg$(interp V p)

5.3.2 Conjunctive Normal Form (CNF)

CNF requires the formula to be a conjunction of clauses, where each clause is a disjunction of literals. Converting to CNF involves distributing disjunctions over conjunctions: $A \vee (B \wedge C) \to (A \vee B) \wedge (A \vee C)$.

I implemented a helper function `distribute` and proved it correct.

```
Lemma distribute_correct : ∀ p1 p2 V,
interp V (distribute p1 p2) = interp V (FDisj p1 p2).
```

Using this, we defined `to_cnf` and proved that the entire transformation preserves semantics. Finally, we integrated this into the solver, defining a `solver_cnf` and proving it is also sound and complete.

6 Discussion and Conclusion

This project illustrates the rigorous process of verifying a critical software component.

6.1 Challenges and Solutions

The most significant friction point was managing the "bookkeeping" of lists in Rocq. In informal mathematics, we easily say "let V be a valuation." In Rocq, establishing that V exists within a generated list required intricate lemmas about `map`, `In`, and list inclusion. I solved this by creating modular helper lemmas (like `in_map_iff_override`) rather than attempting to prove everything in one monolithic block. This modularity made the final completeness proof readable and manageable.

6.2 Experience with Rocq

My experience with Rocq was a mix of frustration and satisfaction. **What we liked:** The rigorous feedback loop. In standard programming, you hope your edge cases are covered. In Rocq, the system forces you to handle every case (e.g., the empty list case in `all_valuations`). The ‘induction‘ tactic is incredibly powerful and aligns well with how computer scientists think about recursive data structures. **What we did not like:** The verbosity of trivial logical steps. Proving simple arithmetic or boolean identities sometimes required multiple lines of tactics, which

felt like efficiently handling "low-level" logic while trying to focus on "high-level" algorithms.

6.3 Conclusion

The resulting artifact is a fully verified SAT solver. While it uses a naive exponential-time algorithm, the proofs of soundness and completeness are valid regardless of the algorithm's speed. This project lays the groundwork for verifying more efficient algorithms (like DPLL) by establishing the necessary semantic framework and fundamental lemmas about boolean logic.

6.4 Use of GAI

Generative AI tools were employed in this project to enhance the quality of the written presentation and to assist in code refactoring. Specifically, they were used to ensure a high standard of academic English throughout the report and to optimize specific proofs for better readability and efficiency, streamlining the tactic scripts while maintaining formal correctness.

A Appendix: Code Fragments

A.1 Optimization Logic

```
Fixpoint optimize (p : form) : form :=
match p with
| var x ⇒ var x
| FTrue ⇒ FTrue
| FFalse ⇒ FFalse
| FNeg p' ⇒
match optimize p' with
| FTrue ⇒ FFalse
| FFalse ⇒ FTrue
| opt_p ⇒ FNeg opt_p
end
(* ... cases for Conj, Disj ... *)
end.
```

A.2 Distribution Logic for CNF

```
Fixpoint distribute (p1 p2 : form) : form :=
match p1 with
| FConj a b ⇒ FConj (distribute a p2) (distribute b p2)
| _ ⇒
(fix dist_helper (p2_inner : form) : form :=
match p2_inner with
| FConj c d ⇒ FConj (dist_helper c) (dist_helper d)
| _ ⇒ FDisj p1 p2_inner
end) p2
end.
```